

Memoria Técnica Aplicación



**UNIVERSIDAD
DE GRANADA**

Santiago López Cerro
Mario Lindez Martinez
Antonio Javier Rodríguez Romero
Diego Ortega Sanchez
Jaime Corzo Galdó

1. Índice

1. Índice.....	2
2. Software.....	3
2.1. Librerías Externas.....	3
2.2. Descripción de clases.....	3
2.3. Estructura del proyecto.....	4
3. Hardware para testing.....	5
4. Sensores utilizados.....	6
5. Reparto de Trabajo.....	7

2. Software

Para la creación de la aplicación móvil se ha hecho uso del entorno **Android Studio**, más concretamente de su versión 2025.2.1. Dentro de este, se ha programado en **Kotlin**, lenguaje de programación optimizado para ejecución y acceso a sensores en Android.

Por otro lado, el nivel mínimo de API para la ejecución de la aplicación en un dispositivo es **SDK 24**. Este se corresponde con la versión comercial **Android 7.0 (Nougat)**.

2.1. Librerías Externas

Enumeramos las librerías externas que se han utilizado para este proyecto:

- **ZXing Android Embedded**: Integración rápida para el escaneo y decodificación de códigos QR y barras.
- **Kotlin Coroutines**: Gestión eficiente de tareas asíncronas y concurrencia (procesos en segundo plano).
- **Google Material Components**: Biblioteca de componentes de interfaz de usuario (UI) basados en Material Design.
- **AndroidX Lifecycle KTX**: Extensiones de Kotlin para gestionar el ciclo de vida de la aplicación dentro de las corrutinas (lifecycleScope).
- **AndroidX AppCompat & Core**: Componentes base para garantizar la compatibilidad con versiones antiguas de Android.
- **Android Beacon Library (AltBeacon)**: Librería para detectar, monitorear y estimar la distancia de balizas Bluetooth LE (Beacons).
- **Gson (Google)**: Librería para convertir objetos Java/Kotlin a formato JSON y viceversa (serialización y deserialización).

Por otro lado, aunque no son externas, han sido de máxima relevancia las siguientes:

- **Android NFC API** (android.nfc): Utilizado para la lectura de etiquetas NFC.
- **Android Sensor API** (android.hardware.Sensor): Utilizado para acceder al hardware (acelerómetros, etc.).

2.2. Descripción de clases

A continuación se enumeran las clases principales del proyecto y se describe brevemente la funcionalidad de cada una.

- **HomeActivity**: Controlador principal que orquesta la interfaz, gestión de sensores en tiempo real y navegación global.
- **BeaconRepository**: Gestiona el escaneo en segundo plano de balizas Bluetooth LE para identificar la sala y ubicación actual.
- **BarometerRepository**: Abstrae la lectura del sensor de presión (barómetro) para permitir el cálculo de altitud en interiores.

- **CurrentFloorResolver**: Deduce la planta en la que se encuentra el usuario a partir de información que introduzca el usuario y sensores.
- **UserPreferencesRepository**: Gestiona la persistencia de datos local (calibración, modo de visita y configuración) mediante DataStore.
- **ShakeDetector**: Algoritmo que procesa los eventos del acelerómetro para detectar el gesto físico de agitación ("shake").
- **BackgroundUtils**: Clase utilitaria que resuelve dinámicamente los recursos gráficos (fondos) según el POI detectado.
- **MapActivity**: Responsable de renderizar los planos interactivos del edificio y situar al usuario en ellos.
- **PoiDetailActivity**: Visualiza el contenido educativo multimedia (texto, imágenes) asociado a un punto de interés detectado.
- **ConfirmationActivity**: Pantalla intermedia para validar el resultado de un escaneo de código QR o etiqueta NFC.
- **CalibrationActivity**: Interfaz para ajustar la referencia de altitud del barómetro manualmente.

2.3. Estructura del proyecto

El proyecto se organiza bajo el paquete raíz **com.example.npi_planetario_movil** siguiendo una arquitectura modular que desacopla la lógica de negocio de la interfaz de usuario.

```

/com.example.npi_planetario_movil
├── /beacon                # Gestión de bajo nivel para protocolos de balizas.
├── /core                  # Núcleo de la aplicación (Lógica compartida y Datos)
│   ├── /model             # Modelos de datos.
│   ├── /repository        # Patrón Repositorio para gestión de datos (Sensores, BD, Prefs).
│   └── [Utils]            # Clases de utilidad (CurrentFloorResolver, BackgroundUtils...).
├── /feature               # Capa de Presentación organizada por módulos funcionales
│   ├── /barometer         # Calibración y gestión del barómetro.
│   ├── /home              # Pantalla principal (Dashboard y control central).
│   ├── /mode              # Selección de modos de visita.
│   ├── /poi               # Visualización de detalles de Puntos de Interés.
│   ├── /scan_qr           # Lógica de escaneo de códigos QR y confirmación.
│   ├── /myth              # Panel de información de Dioses Griegos y su minijuego
│   ├── /warehouse         # Lógica de la información del almacén.
│   ├── /games             # Lógica del panel de juegos
│   ├── /web               # Lógica para manejar contenidos usando WebView
│   ├── /language          # Referente a la lógica detrás de la localización
│   └── /urano             # Minijuego de la sala de Urano.
├── /ui.theme              # Definición de temas, colores y tipografías.
└── /assets
    └── /html               # Contenidos html.
  
```

```
└─ games.json      # Fichero de definición de juegos disponibles.
└─ content.json    # Fichero de definición de puntos de interés.
```

Los paquetes principales son:

- **core:** Contiene la lógica transversal de la aplicación. Aquí residen los Repositorios (BeaconRepository, BarometerRepository, etc.) que actúan como única Knowledge Base para los datos, y los Resolvers (CurrentFloorResolver) que procesan la lógica compleja de sensores.
- **feature:** Contiene las Activities y la lógica de interfaz (UI). Cada carpeta representa una pantalla o flujo de usuario completo, lo que facilita el mantenimiento y la escalabilidad.

3. Hardware para testing

Santiago López Cerro - OPPO A53s	
Procesador	Qualcomm SM4250 Ocho núcleos
Modelo	CPH2135
RAM	4 GB
Versión de Android	12
Dimensiones	6.5"

Jaime Corzo Galdó - Galaxy A56 5G	
Procesador	Samsung Exynos 1580
Modelo	SM-A566B/DS
RAM	8 GB
Versión de Android	16
Dimensiones	6,7"

Diego Ortega Sánchez - Google Pixel 8 Pro	
Procesador	Google Tensor G3
Modelo	Nona-core (1x3.0 GHz Cortex-X3 & 4x2.45 GHz Cortex-A715 & 4x2.15 GHz Cortex-A510)

RAM	12 GB
Versión de Android	16
Dimensiones	6,7”

Mario Lindez Martínez - LG Velvet 5G	
Procesador	Snapdragon 765G 2.4GHz
Modelo	G900TM
RAM	6 GB
Versión de Android	Android 13.0
Dimensiones	6.8”

4. Sensores utilizados

A continuación se enumeran los diferentes sensores de los que hará uso la aplicación móvil, la API de Android que utiliza para interactuar con ellos y sus funcionalidades principales dentro del proyecto.

- **Acelerómetro** (`Sensor.TYPE_ACCELEROMETER`): detección de movimientos bruscos (gesto “Shake”) para invocar el atajo al mapa de la planta.
- **Barómetro** (`Sensor.TYPE_PRESSURE`): medición de la presión atmosférica para calcular la altitud y, a partir de esta, cambios de planta del museo.
- **Giroscopio** (`Sensor.TYPE_GYROSCOPE`): control de rotación y balanceo del dispositivo preciso en juegos internos de la aplicación que hacen uso de esta mecánica.
- **Bluetooth LE** (`BluetoothAdapter/AltBeacon`): Escaneo para la detección de balizas (“Beacons”) utilizadas para determinar la localización.
- **NFC** (`NfcAdapter`): lectura de etiquetas de proximidad para acceder a más información sobre distintos elementos del museo.

- **Cámara** (Camera vía ZXing): captura óptica y escaneo para la lectura de códigos QR que derivarán en información sobre salas o elementos de mucha afluencia en el museo.

5. Reparto de Trabajo

Santiago López Cerro	Implementación de la tecnología NFC, especialmente en la parte del frontend, en lo relativo a su diseño y funcionalidad para adaptarlo a las visitas guiadas que presenta el museo. Implementando de esta misma manera diversos ejemplos de prueba para cada visita. Descripción de la aplicación y su implementación en la memoria del museo junto con otros cambios asociados. Ajuste de Beacons mediante el usos de dispositivos BlueTooth
Mario Líndez Martínez	Implementación de la tecnología NFC, especialmente en la parte backend. Estructura para aquellas personas que no tengan opción de NFC y QR. Mejora en la interfaz gráfica. Respecto a los dispositivos Beacon también he diseñado diversas mejoras, como por ejemplo la visualización de la sala en la que se encuentra el usuario. Ajustes en la implementación del barómetro y posterior calibrado. Integración del visor HTML WebView2 en la aplicación.
Antonio Javier Rodríguez Romero	Desarrollo de sistema de localización interior por medio de Beacons BLE. Diseño de HTMLs informativos de cada uno de los planetas y para la representación de modelos 3D de estos. Descripción del proyecto en la memoria técnica de la aplicación.
Diego Ortega Sánchez	Desarrollo de las siguientes funcionalidades. Función “Estoy perdido” al agitar el dispositivo desde Home. Sala

	interactiva de la Antigüedad (planetas como dioses). Desarrollo del juego con giroscopio de dicha sala. Colaboración en frontend del juego de aterrizaje Urano.
Jaime Corzo Galdó	Me he encargado de la configuración del sistema de idiomas de la aplicación y de la integración de los mapas. También he implementado la lógica de lectura y procesamiento de códigos QR, la extracción de IDs con NFC y el uso del barómetro, además de la gestión de permisos del dispositivo (ubicación, cámara y Bluetooth). En cuanto a la interfaz, he diseñado y programado la pantalla del menú principal, la pantalla de selección del tipo de visita y la pantalla de elección de idioma. Además, he participado en el desarrollo del minijuego de Urano. Mi trabajo se ha centrado en asegurar tanto el correcto funcionamiento técnico como una experiencia de usuario clara y sencilla.

Todas las decisiones de diseño a nivel de estética y funcionalidades de la aplicación, incluyendo los sensores y funcionalidades que utilizaría han sido fruto de la puesta en común de todos los componentes del grupo en reuniones periódicas durante el desarrollo del proyecto.